Digital Pedon

v 2.0.0

User Manual & Getting Started Guide

A Digital Twin Framework for Soil Profile Monitoring, Physics-Based Inference, and Global Interoperability

Dr. Ir. Ali Youssef[1,2] · Dr. Ir. Nasem Badreldin[1,3]

[1] Agroecosystems Laboratory, University of Manitoba, Winnipeg, Canada

[2] DigitalTwinRs Limited

[3] Grassland Analytica Limited

Ali Youssef: ayoussefgm@gmail.com · ali.youssef@umanitoba.ca

Nasem Badreldin: Nasem.Badreldin@umanitoba.ca

Version 2.0.0 · March 2026 · CC BY 4.0

github.com/Pierianspring/digital-pedon

About This Document

This manual provides a complete reference for installing, configuring, and using the Digital Pedon (DP) framework. It is organised into two parts: a **Getting Started Guide** that takes a new user from installation to a running pedon in under 10 minutes, and a **Reference Manual** that documents every public function, configuration key, and output field. No prior soil physics knowledge is required to follow the Getting Started section; the Reference section assumes familiarity with basic soil science concepts.

How to use this manual

New user? Start at Section 1 (Installation) and work through Section 3 (Quick Start).
Looking for a specific function? Jump to Section 5 (API Reference).
Connecting your own sensors? See Section 4.3 (BYOD Sensor Manifest).
Registering a custom solver? See Section 4.4 (Custom Solvers).
Trouble? See Section 7 (Troubleshooting).

1. Installation

1.1 Requirements

Digital Pedon has **zero runtime dependencies** — it runs on the Python standard library only. The only requirement is Python 3.9 or later.

Requirement	Version	Notes
Python	≥ 3.9	Standard library only — no numpy, pandas, or scipy required
PyYAML	≥ 6.0	Optional — only needed for YAML sensor manifests ([yaml] extra)
Anthropic SDK	≥ 0.20	Optional — only needed for LLM agent examples ([llm] extra)
matplotlib	≥ 3.5	Optional — only needed to run the experiment plotting scripts
numpy	≥ 1.21	Optional — only needed for the heat flux experiment (Experiment 4)

1.2 Install from GitHub

Open a terminal (VS Code terminal on Mac/Windows, or any shell) and run:

```
# Clone the repository
git clone https://github.com/Pierianspring/digital-pedon.git
cd digital-pedon

# Create and activate a virtual environment (recommended)
python3 -m venv .venv
source .venv/bin/activate        # Mac / Linux
.venv\Scripts\activate           # Windows

# Install the package (editable mode)
pip install -e .

# Optional extras
pip install -e .[yaml]            # adds PyYAML for YAML manifests
pip install -e .[dev]             # adds pytest for running tests
```

> *On VS Code (Mac): after creating the venv, press Cmd+Shift+P → 'Python: Select Interpreter' → choose the .venv entry. VS Code will then activate the venv automatically in every new terminal.*

1.3 Verify the installation

```
python -c "from digital_pedon import build_pedon; print('Digital Pedon OK')"
```

Expected output: `Digital Pedon OK`

1.4 Run the test suite

Before using the framework with your own data, confirm that all 95 automated tests pass:

```
pip install -e .[dev]     # install pytest first
pytest tests/ -v
```

You should see **95 tests passing** across three test files. If any fail, check your Python version (must be ≥ 3.9).

2. Core Concepts

2.1 The Digital Pedon

A **Digital Pedon** is a continuously updated computational representation of a multi-horizon soil profile. Unlike a static soil database record, a DP ingests real-time sensor observations, computes derived physical quantities on every update, fires agronomic threshold alarms, and maintains a full time-series history. The key design principle is *dictionary-first*: every input and output is a plain Python dictionary — no subclassing, no framework-specific objects, no imports beyond the function you are calling.

2.2 The Three-Layer Architecture

Structural Layer (SL) — the skeleton

Set once at build_pedon() time, never modified at runtime.
Stores: horizon geometry (depth, FAO designation), texture, organic carbon, bulk density, pH, CEC, Van Genuchten parameters, threshold configuration.
Key function: build_pedon(config_dict)

Dynamic Layer (DL) — live state

Updated on every sensor reading. Three internal data structures:

- Key-Value Store (KVS): current state per horizon, O(1) access.
- Time-Series Log (TSL): append-only history, queryable by time range.
- Threshold Event System: fires 5 agronomic alarms automatically.

Key functions: pedon.update_sync(), pedon.query_history()

Functional Layer (FL) — physics solvers (Model Zoo)

Runs all registered solvers in sequence on every update() call.
Built-in solvers: Van Genuchten–Mualem, Darcy–Buckingham, De Vries heat, Q10 respiration.
Add any Python function as a solver with register_method().
Key function: register_method(method_dict)

2.3 Dictionary-First Design

Everything in the DP is a plain Python `dict`. There are no classes to instantiate, no schema files to import, and no mandatory field names beyond three geometric fields per horizon. Any key you add to a horizon dictionary is automatically stored and passed to all registered solvers. This means you can add a new soil property — say `microplastic_load_mg_kg` — without any code changes to the framework.

3. Getting Started — First Pedon in 5 Minutes

This section walks through building a complete two-horizon pedon, feeding a sensor reading, and inspecting the physics output. Run from inside the `digital-pedon` folder with the venv activated.

3.1 Build your first pedon

```
from digital_pedon import build_pedon, print_snapshot

pedon = build_pedon({
    "site_name": "My First Site",
    "latitude":   51.05,
    "longitude":   3.72,
    "thresholds": {
        "theta_saturation": 0.44,   # fires saturation alarm above this
        "theta_wilting_pt":  0.09,   # fires wilting point alarm below this
    },
    "horizons": [
        {
            "designation":    "Ap",
            "depth_top_cm":   0,
            "depth_bottom_cm": 28,
            "soil_type":      "loamy_topsoil",  # uses built-in template
        },
        {
            "designation":    "Bw",
            "depth_top_cm":   28,
            "depth_bottom_cm": 70,
            "soil_type":      "clay_subsoil",
        },
    ],
})
print(pedon)
```

Output: `DigitalPedon(id='...', site='My First Site', horizons=2, methods=['van_genuchten', 'heat_equation', 'carbon_flux_Q10'])`

3.2 Feed a sensor reading

```
result = pedon.update_sync({
    "horizon_id":                "Ap",
    "volumetric_water_content":  0.31,   # cm3/cm3
    "soil_temperature_c":        14.2,   # degC
    "electrical_conductivity_ds_m": 0.4, # dS/m
})

# All four solvers ran automatically. Read the results:
print(result["theta"])                                  # 0.31
print(result["derived"]["matric_potential_cm"])          # -287.4
print(result["derived"]["hydraulic_conductivity_cm_day"]) # 0.023
print(result["derived"]["soil_respiration_mg_C_cm3_day"]) # 0.058
```

3.3 Print a full snapshot

```
print_snapshot(pedon)
```

This prints a formatted table of all horizons with their current state (θ, T, EC, ψ, q) and derived quantities.

3.4 Query the history

```
# Get all readings for the Ap horizon
history = pedon.query_history("Ap")
print(f"{len(history)} readings stored")
print(history[-1])   # most recent reading

# Filter by time range (Unix timestamps)
import time
since = time.time() - 3600   # last hour
recent = pedon.query_history("Ap", since=since)
```

3.5 Export as JSON-LD

```
# Self-describing export with GLOSIS URIs in @context
json_str = pedon.to_json()
print(json_str[:400])   # preview

# Save to file
with open("my_pedon.jsonld", "w") as f:
    f.write(json_str)
```

4. Detailed Usage

4.1 Initialising from Global Databases

If you have GPS coordinates, the DP can fetch all soil properties automatically from the SoilGrids v2.0 REST API and estimate Van Genuchten parameters using Rawls-Brakensiek pedotransfer functions:

```
from digital_pedon import build_pedon, print_snapshot
from digital_pedon.sources import fetch_soil_profile

# Fetch from SoilGrids using GPS coordinates
config = fetch_soil_profile(latitude=51.05, longitude=3.72)

# Override one value with your own lab measurement
for hz in config["horizons"]:
    if hz.get("designation") == "Ap":
        hz["organic_carbon_pct"] = 3.8   # your lab value wins

pedon = build_pedon({**config, "site_name": "Ghent Field Site"})
print_snapshot(pedon)
```

> *SoilGrids is a free public API and may be temporarily unavailable. The framework handles this gracefully — if the API returns a 503 error, a clear message is printed and you can fall back to manual initialisation.*

4.2 Built-in Soil Types

Five soil type templates are built into the framework. When you set `"soil_type": "loamy_topsoil"`, all properties from the template are copied as defaults, which you can override individually:

Soil type key	Description	Clay %	Sand %	θs	Ks (cm/d)
loamy_topsoil	Typical loam A-horizon	18	40	0.480	15.0
clay_subsoil	Heavy clay B-horizon	48	20	0.460	1.0
sandy_loam	Sandy loam	10	65	0.410	35.0
peat	Organic topsoil	5	15	0.800	5.0
calcareous_loam	$CaCO_3$-rich loam	22	38	0.470	12.0

To register your own soil type:

```
from digital_pedon import register_soil_type

register_soil_type("my_clay", {
    "clay_pct": 55, "sand_pct": 15, "silt_pct": 30,
    "bulk_density_g_cm3": 1.45,
    "organic_carbon_pct": 0.8,
    "theta_r": 0.075, "theta_s": 0.52,
    "alpha_vg": 0.007, "n_vg": 1.08, "Ks_cm_day": 0.5,
})
```

4.3 BYOD Sensor Manifest

The Bring-Your-Own-Device (BYOD) sensor manifest lets you declare your sensors once and ingest any logger output automatically, with unit conversion applied transparently:

```
from digital_pedon.sensor import ManifestBuilder, SensorLayer

manifest = (
    ManifestBuilder("Field Site A", "SITE_001")

    # Ap horizon — TEROS 11, reading VWC in %, temperature in degF
    .add_insitu(source_key="VWC_15cm_pct",
                canonical_key="volumetric_water_content",
                unit="%",        # auto-converted to cm3/cm3
                horizon_id="Ap", depth_cm=15, sampling_rate_s=3600,
                sensor_type="METER TEROS 11")

    .add_insitu(source_key="Temp_15cm_F",
                canonical_key="soil_temperature_c",
                unit="degF",     # auto-converted to degC
                horizon_id="Ap", depth_cm=15, sampling_rate_s=3600)

    .add_insitu(source_key="EC_uScm",
                canonical_key="electrical_conductivity_ds_m",
                unit="uS/cm",    # auto-converted to dS/m
                horizon_id="Ap", depth_cm=15, sampling_rate_s=3600)
    .build()
)

# Parse one row from your logger CSV
raw_row = {"VWC_15cm_pct": 34.1, "Temp_15cm_F": 57.6, "EC_uScm": 420}
reading  = manifest.parse_row(raw_row)  # units converted automatically
result   = pedon.update_sync(reading)
```

Supported unit conversions:

Source unit	Canonical unit	Conversion
%	cm^3/cm^3	÷ 100
m3/m3	cm^3/cm^3	1:1 (no change)
degF	°C	(F − 32) × 5/9
K	°C	− 273.15
uS/cm	dS/m	÷ 1000
mS/cm	dS/m	1:1 (no change)
mS/m	dS/m	÷ 100
kPa	cm H_2O	× 10.197
hPa	cm H_2O	× 1.0197
bar	cm H_2O	× 1019.7
pF	cm H_2O	−10^pF

MPa	cm H_2O	× 10197
DN (S2)	reflectance	÷ 10000
mV	mV	1:1 (EC sensor passthrough)

4.4 Registering a Custom Solver

Any Python function with the signature `fn(params: dict, state: dict) -> dict` can be registered as a solver. It runs automatically on every `update()` call alongside the built-in solvers, and its output is merged into the state dictionary for downstream solvers.

```
from digital_pedon import register_method

def plant_available_water(params, state):
    """PAW fraction between wilting point and field capacity."""
    theta    = state.get("theta", 0)
    theta_fc = params.get("theta_s", 0.40) * 0.70   # rough field capacity
    theta_wp = params.get("theta_r", 0.08) * 1.20
    paw = max(0.0, min(1.0, (theta - theta_wp) / max(theta_fc - theta_wp, 1e-
6)))
    return {
        "plant_available_water_fraction": round(paw, 3),
        "water_stress": paw < 0.20,
    }

register_method({
    "name":        "plant_available_water",
    "fn":           plant_available_water,
    "description": "PAW fraction between wilting point and field capacity",
    "inputs":      ["theta", "theta_s", "theta_r"],
    "outputs":     ["plant_available_water_fraction", "water_stress"],
})

# Now runs automatically — access output from result["derived"]
result = pedon.update_sync({"horizon_id": "Ap", "volumetric_water_content":
0.25})
print(result["derived"]["plant_available_water_fraction"])  # e.g. 0.73
```

4.5 Threshold Events

Five agronomic alarms fire automatically when conditions are met. You can register a callback function that runs on every alarm:

Event name	Condition	Default threshold
saturation	θ ≥ θsat	0.44 cm³/cm³
wilting_point	θ ≤ θwp	0.09 cm³/cm³
high_temp	T ≥ T_high	35.0 °C
low_temp	T ≤ T_low	0.0 °C
high_ec	EC ≥ EC_high	4.0 dS/m

```
def my_alert(horizon_id, event_name, state):
    print(f"ALERT [{horizon_id}]: {event_name}")
    print(f"  Current state: theta={state['theta']:.3f},
T={state['temperature_c']:.1f}C")

pedon.register_event_handler(my_alert)

# Overrides at build time:
pedon = build_pedon({
    ...
    "thresholds": {
        "theta_saturation": 0.44,
        "theta_wilting_pt":  0.09,
        "temp_high_c":       35.0,
        "temp_low_c":         2.0,
        "ec_high_ds_m":       4.0,
    },
})
```

5. API Reference

5.1 Top-Level Functions

Function	Returns	Description
`build_pedon(config)`	`DigitalPedon`	Build and return a digital twin from a config dict
`register_method(method_dict)`	`None`	Register a custom solver in the global Model Zoo
`unregister_method(name)`	`None`	Remove a solver from the registry
`list_methods()`	`list[dict]`	Return metadata for all registered solvers
`register_soil_type(name, d)`	`None`	Add a new soil type template to the registry
`print_snapshot(pedon)`	`None`	Pretty-print the current state of all horizons

5.2 DigitalPedon Object Methods

Method	Returns	Description
`update_sync(reading_dict)`	`dict`	Ingest one sensor reading; run all solvers; return horizon state
`snapshot()`	`dict`	Return the full current state of all horizons
`query_history(hz_id, since, until, n)`	`list[dict]`	Return time-series history for one horizon
`register_event_handler(fn)`	`None`	Register a callback for threshold events
`get_horizon_state(horizon_id)`	`dict`	Return the current state dict for one horizon
`to_json(indent=2)`	`str`	Serialise the full snapshot to JSON-LD string

5.3 update_sync() Input Keys

The reading dict passed to `update_sync()` must contain `horizon_id` plus any measured values. Both canonical names and legacy aliases are accepted:

Canonical key	Alias	Unit	Description
horizon_id	—	—	FAO designation (e.g. 'Ap') or internal UUID
volumetric_water_content	theta	cm^3/cm^3	Volumetric soil water content
soil_temperature_c	temperature_c	°C	Soil temperature
electrical_conductivity_ds_m	ec_ds_m	dS/m	Bulk electrical conductivity
matric_potential_cm	psi	cm H_2O	Matric potential (optional override)
timestamp	—	Unix s	Reading timestamp (default: now)

5.4 update_sync() Output Keys

The dict returned by `update_sync()` contains the following top-level keys:

Key	Type	Description
theta	float	Volumetric water content (cm^3/cm^3)
temperature_c	float	Soil temperature (°C)
ec_ds_m	float	Electrical conductivity (dS/m)
matric_potential_cm	float	Matric potential from Van Genuchten (cm H_2O)
flux_down_cm_day	float	Darcy vertical flux (cm/day; + = downward)
last_updated	float	Unix timestamp of this reading
derived	dict	All quantities computed by solvers (see below)

The `derived` sub-dict contains the outputs of all registered solvers:

derived key	Solver	Unit
matric_potential_cm	Van Genuchten	cm H_2O
effective_saturation	Van Genuchten	—
rel_hydraulic_conductivity	Van Genuchten	—

hydraulic_conductivity_cm_day	Van Genuchten	cm/day
heat_capacity_J_cm3_K	De Vries	J/cm³/K
thermal_diffusivity_cm2_day	De Vries	cm²/day
soil_respiration_mg_C_cm3_day	Q10	mg C/cm³/day
+ any custom solver outputs	User-defined	User-defined

6. Configuration Reference

6.1 build_pedon() Config Keys

Key	Required	Description
site_name	No	Human-readable site name (default: 'Unnamed Site')
latitude	No	WGS84 latitude — enables SoilGrids fetch
longitude	No	WGS84 longitude — enables SoilGrids fetch
horizons	Yes	List of horizon dicts (see Section 6.2)
thresholds	No	Override default alarm thresholds (see Section 4.5)

6.2 Horizon Dictionary Keys

Key	Required	Description
designation	Yes	FAO/WRB morphological code: Ap, Bw, Btg, C, etc.
depth_top_cm	Yes	Upper horizon boundary (cm from surface)
depth_bottom_cm	Yes	Lower horizon boundary (cm from surface)
soil_type	No	Built-in template name — sets all properties as defaults
clay_pct	No	Clay mass fraction (%) — overrides soil_type value
sand_pct	No	Sand mass fraction (%)
silt_pct	No	Silt mass fraction (%)
organic_carbon_pct	No	Organic carbon (%)
bulk_density_g_cm3	No	Dry bulk density (g/cm³)
ph	No	pH in water (1:2.5)
theta_r	No	Van Genuchten residual water content (cm³/cm³)
theta_s	No	Van Genuchten saturated water content (cm³/cm³)
alpha_vg	No	Van Genuchten α parameter (1/cm)
n_vg	No	Van Genuchten n shape parameter
Ks_cm_day	No	Saturated hydraulic conductivity (cm/day)
[any other key]	No	Stored and passed to all solvers automatically

7. Troubleshooting

Error / symptom	Cause	Fix
ModuleNotFoundError: digital_pedon	Package not installed or wrong directory	Run pip install -e . from inside the digital-pedon folder
KeyError: 'volumetric_water_content'	Accessing wrong key in result dict	Use result['theta'] not result['volumetric_water_content']
KeyError: 'matric_potential_cm'	Van Genuchten params missing for this horizon	Add soil_type or provide theta_r, theta_s, alpha_vg, n_vg
HTTP 503 from SoilGrids	ISRIC REST API temporarily down	Wait 30-60 min and retry; use offline fallback meanwhile
ValueError: left cannot be >= right	GridSpec layout error (matplotlib)	Check gs_left < gs_right in the STYLE block
All threshold alarms fire immediately	theta value in percent, not cm3/cm3	Use 0.35 not 35 — or declare unit='%' in the manifest
pytest: 0 tests collected	Wrong directory or missing pytest	Run pip install -e .[dev] then pytest tests/ -v from digital-pedon/
git: Authentication failed	GitHub no longer accepts passwords	Use a Personal Access Token from github.com/settings/tokens

8. Examples Index

The **`examples/`** folder contains five ready-to-run scripts:

File	Run command	What it demonstrates
`01_quick_start.py`	`python examples/01_quick_start.py`	Build pedon, register solver, feed reading, inspect results
`02_soilgrids_fetch.py`	`python examples/02_soilgrids_fetch.py`	GPS initialisation from SoilGrids + offline fallback
`03_sensor_manifest.py`	`python examples/03_sensor_manifest.py`	BYOD manifest, unit conversion, batch CSV ingestion
`04_remote_sensing.py`	`python examples/04_remote_sensing.py`	NDVI, LST, SAR soil moisture index ingestion
`05_llm_agent.py`	`python examples/05_llm_agent.py`	LLM agent querying the soil (requires Anthropic SDK)

9. Citation

If you use Digital Pedon in your research, please cite:

How to cite

Youssef, A. & Badreldin, N. (2026). DigitalPedon: A Digital Twin Framework for Soil Profile Monitoring and Global Interoperability.
Version 2.0.0. CC BY 4.0.
https://github.com/Pierianspring/digital-pedon

BibTeX:

```
@software{digitalpedon2026,
  author  = {Youssef, Ali and Badreldin, Nasem},
  title   = {Digital Pedon v2.0.0},
  year    = {2026},
  license = {CC-BY-4.0},
  url     = {https://github.com/Pierianspring/digital-pedon}
}
```

10. Licence

Digital Pedon is released under the **Creative Commons Attribution 4.0 International (CC BY 4.0)** licence. You are free to use, share, and adapt this software for any purpose, including commercial use, provided you give appropriate credit to the original authors.

Full licence text: https://creativecommons.org/licenses/by/4.0/